Gen AI Key Models and There Use cases

Hobhav S Fulzele

27 octobre 2026

Contents

1 Key Models **3**
- 1.1 GPT . . . 3
- 1.2 DALL-E . . . 5
- 1.3 Codex . . . 7
- 1.4 Stable Diffusion . . . 8

2 What Is Diffusion ? **10**
- 2.1 Basic Concept . . . 11
- 2.2 Forward Diffusion Process . . . 11
- 2.3 Reverse Diffusion Process . . . 11
- 2.4 Training Objective . . . 12
- 2.5 Text-Conditioned Diffusion . . . 12
- 2.6 Applications of Diffusion Models . . . 12
- 2.7 Advantages and Limitations . . . 13

3 What Are GANs ? **13**
- 3.1 Main Components of a GAN . . . 13
- 3.2 How GANs Work . . . 13
- 3.3 Mathematical Formulation . . . 14
- 3.4 Applications of GANs . . . 14
- 3.5 Advantages and Limitations . . . 15

4 What Are Transformers ? **15**
- 4.1 Why Transformers Were Developed . . . 15
- 4.2 Main Components of a Transformer . . . 15
- 4.3 Encoder and Decoder Architectures . . . 17
- 4.4 Causal Attention . . . 17
- 4.5 Applications of Transformers . . . 17
- 4.6 Advantages and Limitations . . . 18

5 Applications in Enterprise AI **18**
5.1 Customer Service and Support 18
5.2 Document Processing and Knowledge Management 18
5.3 Software Development and IT Operations 19
5.4 Human Resources and Employee Productivity 19
5.5 Finance and Business Analysis 20
5.6 Supply Chain and Manufacturing 20

1 Key Models

Generative Artificial Intelligence (Generative AI) refers to a category of artificial intelligence systems capable of creating new content based on patterns learned from training data. The generated content may include text, images, source code, audio, video, and other forms of digital information.

Traditional AI systems are often designed to classify data, predict outcomes, or identify patterns. In contrast, generative AI models learn the underlying structure of their training data and use that knowledge to produce new outputs.

Generative AI has become an important technology in areas such as software development, healthcare, education, marketing, scientific research, customer service, and enterprise automation.

Some important generative AI models and systems include GPT, DALL-E, Codex, and Stable Diffusion. These systems have different design goals and capabilities, but they share the objective of using learned patterns to generate useful outputs.

1.1 GPT

GPT stands for *Generative Pre-trained Transformer*. It is a family of language models developed by OpenAI and based on the Transformer neural network architecture.

GPT models are designed primarily to understand and generate natural language. Depending on the model and its capabilities, they can also support tasks such as summarization, translation, reasoning, information extraction, question answering, and multimodal interaction.

1.1.1 Meaning of GPT

The three components of GPT describe its general approach :

- **Generative :** The model produces new sequences of tokens, such as words, punctuation, or code.
- **Pre-trained :** The model first learns patterns from large collections of training data before being adapted or optimized for particular tasks.
- **Transformer :** The model uses a neural network architecture that relies on attention mechanisms to represent relationships between tokens in a sequence.

1.1.2 How GPT Works

GPT models typically process text through the following stages.

1. **Tokenization :** The input text is divided into smaller units called tokens. A token may represent a complete word, part of a word, punctuation, or another text unit.
2. **Embedding :** Each token is converted into a numerical vector. Positional information is also incorporated so that the model can represent the order of tokens.

3. **Transformer processing :** The vectors pass through multiple Transformer layers. Attention mechanisms help the model determine which preceding tokens are relevant to the current token.
4. **Probability estimation :** The model estimates a probability distribution over possible next tokens.
5. **Token generation :** A decoding strategy selects the next token. This process is repeated until the response reaches a stopping condition or a specified output limit.

For an autoregressive language model, the probability of a sequence of tokens can be expressed as

$$P(x_1, x_2, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_1, x_2, \ldots, x_{t-1}).$$

Here, x_t represents the token at position t. The equation shows that the model represents a sequence by estimating the probability of each token given the preceding tokens.

During training, the model learns to predict tokens from their context. During generation, it uses the learned probability distribution to produce a continuation of the input.

1.1.3 Training and Adaptation

GPT-style systems generally involve several stages of development and adaptation :

- **Pre-training :** Learning broad linguistic, factual, and structural patterns from large datasets.
- **Instruction tuning :** Training or adapting the model to follow instructions and respond to user requests.
- **Preference optimization :** Using human feedback or other preference signals to improve the usefulness, style, and safety of responses.
- **Inference :** Using the trained model to generate responses to new inputs.

The exact training methods differ between model versions. Not every GPT-based system uses the same training pipeline, and additional tools or retrieval systems may be integrated into a deployed application.

1.1.4 Applications of GPT

GPT models have numerous applications :

- **Content generation :** Creating reports, articles, emails, summaries, and presentations.
- **Customer support :** Answering questions, assisting customers, and helping support teams prepare responses.
- **Education :** Explaining concepts, generating practice questions, and providing personalized learning assistance.

– **Data analysis :** Interpreting tabular data, explaining trends, and helping users write analytical code.

– **Knowledge management :** Searching and summarizing internal documents when connected to approved enterprise information sources.

– **Programming assistance :** Explaining source code, suggesting implementations, and helping identify bugs.

1.1.5 Advantages and Limitations

GPT models can improve productivity, support natural-language interaction, and automate repetitive knowledge-work tasks. They can also adapt to a wide range of tasks without requiring a separate model for every individual use case.

However, GPT models may generate incorrect information, produce biased responses, misunderstand ambiguous instructions, or provide plausible-sounding but unsupported claims. Their outputs should therefore be verified when accuracy is important. Access controls, privacy protections, and appropriate human oversight are particularly important in enterprise applications.

1.2 DALL-E

DALL-E is a family of generative AI systems developed by OpenAI for creating images from natural-language descriptions. It connects language understanding with visual generation, allowing users to describe an image and obtain a corresponding visual result.

For example, a user might request an illustration of a futuristic city at sunset, a product concept for a new device, or a stylized educational diagram. The system interprets the description and generates an image that attempts to satisfy the requested characteristics.

1.2.1 Core Idea

Traditional computer graphics often require a designer to construct visual elements manually. Image-generation systems instead learn statistical relationships between visual content and, in text-conditioned systems, linguistic descriptions.

The model learns associations between concepts such as objects, colors, scenes, materials, styles, and spatial relationships. It uses these associations to generate an image conditioned on a text prompt.

1.2.2 How DALL-E Works

The internal architecture differs across generations of DALL-E. A general description of text-conditioned image generation includes the following stages :

1. **Prompt processing :** The natural-language prompt is analyzed to identify the requested objects, attributes, relationships, and style.

2. **Text representation :** The prompt is represented numerically so that the image generation system can use its semantic information.
3. **Image generation :** The model generates a visual representation conditioned on the prompt. Depending on the generation, this may involve autoregressive image modeling or a diffusion-based approach.
4. **Output refinement :** The system produces the final image according to the capabilities and processing stages of the particular model.

It is important to note that not every DALL-E generation uses the same architecture. Therefore, DALL-E should be understood as a family of image-generation systems rather than a single, unchanging implementation.

1.2.3 Example

Consider the following prompt :

> A modern electric vehicle parked near a glass office building, with green trees, soft morning sunlight, and a realistic photographic style.

The model attempts to represent the requested vehicle, environment, lighting, and artistic style in one coherent image. Changing the prompt can alter the visual output, allowing users to explore different design possibilities.

1.2.4 Applications of DALL-E

- **Marketing and advertising :** Creating campaign concepts, promotional illustrations, and social media visuals.
- **Product design :** Exploring early-stage product concepts, packaging ideas, and visual prototypes.
- **Education :** Generating illustrations that explain scientific, historical, or technical concepts.
- **Entertainment :** Developing concept art, character ideas, and fictional environments.
- **E-commerce :** Supporting visual ideation for product pages, banners, and promotional material.
- **Communication :** Converting abstract ideas into visual representations for presentations and reports.

1.2.5 Advantages and Limitations

DALL-E-style systems can make visual content creation faster and allow people without advanced drawing skills to explore creative ideas. They can generate multiple alternatives from different prompts and support rapid prototyping.

However, generated images may contain incorrect details, inconsistent text, distorted objects, or inaccurate representations of real-world scenes. Users must also consider copyright, licensing, consent, brand guidelines, and the risk of generating misleading imagery.

For enterprise use, generated images should be reviewed before publication, especially when they represent real products, people, technical equipment, or regulated information.

1.3 Codex

Codex is the name of OpenAI's code-focused AI model family and, in current usage, a coding agent experience. The original Codex models were designed to understand and generate programming code, while newer coding systems can support broader software engineering workflows.

Code generation is a specialized application of generative AI because source code has a formal structure, a defined syntax, and behavior that can often be checked by compilers, tests, and execution environments.

1.3.1 Core Idea

A code-focused model learns patterns from programming languages, code examples, and related text. Given an instruction or an existing code fragment, it generates a likely continuation or proposes an implementation.

For example, a developer may request a Python function that sorts a list of records by date, asks for an explanation of an unfamiliar program, or requests help resolving a software bug.

1.3.2 How Code Generation Works

A simplified code-generation workflow consists of the following stages :

1. **Instruction interpretation :** The system identifies the intended functionality, programming language, constraints, and expected output.
2. **Context analysis :** It examines the supplied code, relevant files, function definitions, and other available context.
3. **Code generation :** The model produces a candidate implementation using learned programming patterns.
4. **Validation :** The code may be checked using static analysis, compilation, automated tests, or execution in a controlled environment.
5. **Revision :** Based on errors or test results, the implementation may be revised until it satisfies the requirements or reaches the limits of the workflow.

Modern coding agents may additionally inspect repositories, edit multiple files, run tests, and use development tools. These capabilities depend on the specific system and the permissions granted to it.

1.3.3 Example

Suppose a developer requests a function that calculates the average of a list of numbers. A code-generation system might produce the following Python implementation :

```
def calculate_average(numbers):
    if not numbers:
        return None
    return sum(numbers) / len(numbers)
```

The function handles an empty list explicitly and otherwise calculates the arithmetic mean. A developer should still verify that returning `None` for empty input matches the requirements of the application.

This example demonstrates that code generation involves more than producing syntactically valid code. The generated solution must also satisfy the intended requirements and handle relevant edge cases.

1.3.4 Applications of Codex-Style Systems

- **Code completion :** Suggesting the next line or block of source code.
- **Code explanation :** Describing the purpose and behavior of existing programs.
- **Debugging :** Identifying possible causes of errors and suggesting fixes.
- **Software testing :** Generating unit tests and proposing test cases for edge conditions.
- **Code translation :** Converting code between programming languages or frameworks.
- **Documentation :** Preparing comments, API documentation, and usage examples.
- **Repository-level development :** Supporting multi-file changes, refactoring, and test execution in systems equipped with appropriate development tools.

1.3.5 Advantages and Limitations

Code-generation systems can reduce repetitive programming work, help developers understand unfamiliar codebases, and accelerate prototyping. They may also help beginners learn programming concepts through examples and explanations.

Nevertheless, generated code may contain logical errors, security vulnerabilities, inefficient algorithms, or incorrect assumptions about dependencies and APIs. Code that appears correct may fail under real workloads or unusual inputs.

Organizations should require code review, automated testing, dependency checks, secure development practices, and human approval for consequential changes. AI-generated code should not automatically be treated as production-ready software.

1.4 Stable Diffusion

Stable Diffusion is a family of text-to-image generative models based on latent diffusion techniques. It became widely known for enabling image generation from text prompts, with

implementations and interfaces that can support additional tasks such as image-to-image generation, inpainting, and outpainting.

Unlike approaches that perform the entire diffusion process directly on high-resolution pixels, latent diffusion models perform much of their computation in a compressed latent representation. This can substantially reduce computational requirements compared with operating at full image resolution.

1.4.1 Core Idea

An image contains a large number of pixel values. Directly processing all pixels at every generation step can be computationally expensive.

Stable Diffusion uses a learned representation of an image, called a latent representation. An encoder maps an image into a lower-dimensional latent space, and a decoder reconstructs an image from that representation.

The diffusion model learns to transform noisy latent variables into meaningful latent representations that can be decoded into images.

1.4.2 Main Components

A simplified Stable Diffusion architecture includes three important components :

- **Text encoder :** Converts the text prompt into a numerical representation that guides the generation process.
- **Denoising network :** Often implemented using a U-Net architecture in earlier Stable Diffusion versions, it estimates the noise or related quantities needed to progressively refine the latent representation. Cross-attention can connect the text representation to the visual features.
- **Variational autoencoder (VAE) :** Provides an encoder and decoder for mapping between image space and a compressed latent space.

The exact components and parameterizations vary across model versions, so this description represents a common design rather than every Stable Diffusion implementation.

1.4.3 How Stable Diffusion Works

1. **Prompt encoding :** The input prompt is processed by the text encoder.
2. **Latent initialization :** Generation begins with a noisy latent representation, often sampled from a Gaussian distribution.
3. **Iterative denoising :** A denoising network repeatedly estimates how to update the latent representation. The text prompt guides these updates toward the requested content.
4. **Sampling :** A numerical sampling algorithm determines how the latent representation changes across successive denoising steps.

5. **Image decoding :** The final latent representation is passed through the VAE decoder to produce the output image.

For text-conditioned generation, classifier-free guidance is one technique that can increase adherence to the prompt. A simplified expression for the guided noise estimate is

$$\hat{\epsilon}_\theta(z_t, c) = \epsilon_\theta(z_t,) + s\left[\epsilon_\theta(z_t, c) - \epsilon_\theta(z_t,)\right],$$

where z_t is the noisy latent at step t, c represents the text conditioning, represents an unconditional input, and s is the guidance scale.

A larger guidance scale can strengthen the influence of the prompt, although excessively high values may reduce visual quality or diversity. The equation describes a common guidance method and is not the only possible sampling strategy.

1.4.4 Applications of Stable Diffusion

- **Creative design :** Generating illustrations, concept art, and visual alternatives.
- **Image editing :** Replacing or modifying selected image regions through inpainting techniques.
- **Product visualization :** Exploring packaging, furniture, clothing, and product appearance.
- **Advertising :** Creating draft campaign images and creative concepts.
- **Game development :** Supporting background art, environment concepts, and visual experimentation.
- **Research and prototyping :** Experimenting with prompts, conditioning methods, and image-generation workflows.

1.4.5 Advantages and Limitations

Stable Diffusion implementations can offer flexibility, customization, and opportunities for deployment on suitable local or cloud hardware. Some implementations allow users to adapt models to particular visual styles or domains.

However, resource requirements vary with model size, resolution, sampling settings, and hardware. Outputs may contain visual artifacts, and model behavior depends on training data and fine-tuning. Licensing terms, dataset provenance, privacy, and acceptable-use policies must be reviewed before commercial deployment.

2 What Is Diffusion ?

Diffusion models are a class of generative models that learn to produce data by reversing a process that progressively adds noise to an original sample.

They are widely used in image generation and can also be applied to audio, video, and other data types. Stable Diffusion is a prominent example of an image-generation system based on latent diffusion.

2.1 Basic Concept

The central idea is to train a neural network to recover structure from progressively corrupted data.

For example, consider a clear photograph. If random noise is added repeatedly, the image gradually becomes less recognizable. After enough corruption steps, the resulting sample may resemble random noise.

A diffusion model learns the reverse task : starting from noise, it repeatedly removes or estimates noise to produce a sample resembling the training data distribution.

2.2 Forward Diffusion Process

The forward process gradually adds Gaussian noise to an original data sample x_0. A common mathematical formulation is

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I\right),$$

where :

- x_0 is the original data sample.
- x_t is the noisy sample at time step t.
- β_t is the noise variance schedule.
- I is the identity covariance matrix.
- $\mathcal{N}$ denotes a Gaussian distribution.

The noise schedule determines how much noise is introduced at each step. After many steps, the original structure is progressively obscured.

2.3 Reverse Diffusion Process

The reverse process learns to transform noisy samples into less noisy samples. A neural network estimates the required reverse transition using the current noisy sample and the time step.

A simplified expression is

$$p_\theta(x_{t-1} \mid x_t),$$

where θ represents the trainable parameters of the model.

The model repeatedly applies the learned reverse transitions until it produces a final sample x_0.

For example, generation may begin with random noise and gradually produce an image containing recognizable objects, textures, and scene structure.

2.4 Training Objective

A widely used training objective teaches a network to predict the noise added to a data sample.

The forward process can be expressed directly as

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon,$$

where

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i,$$

and ϵ is sampled from a standard Gaussian distribution.

A common simplified noise-prediction loss is

$$\mathcal{L}_{\text{simple}} = E_{x_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2\right].$$

Here, ϵ_θ is the neural network's estimate of the added noise. The training objective encourages the model to predict the noise accurately across different samples and time steps.

Some diffusion systems use alternative prediction targets, weighting schemes, or objectives, so this equation describes a common formulation rather than every diffusion model.

2.5 Text-Conditioned Diffusion

Text-conditioned diffusion models use a text representation to guide the generation process. A prompt such as *"a red bicycle beside a tree in a park"* provides semantic information about the desired output.

The model uses the prompt representation during denoising to encourage the generated sample to match the requested content.

This approach makes diffusion models useful for text-to-image generation, image editing, and other conditional generation tasks.

2.6 Applications of Diffusion Models

- Image generation from natural-language prompts.
- Image restoration and denoising.
- Image-to-image translation and editing.
- Creation of synthetic training data.
- Audio and speech generation in suitable architectures.
- Video generation and animation in specialized systems.
- Scientific imaging and research applications.

2.7 Advantages and Limitations

Diffusion models can produce high-quality, diverse outputs and offer flexible conditioning mechanisms. They are particularly effective for many image-generation tasks.

Their limitations include inference cost, sensitivity to sampling choices, possible artifacts, and dependence on training data. Multiple denoising steps may be required, although specialized methods can accelerate generation.

type$_i d =$ "$DIFFUSION$"

3 What Are GANs ?

GAN stands for *Generative Adversarial Network*. It is a generative modeling framework introduced by Ian Goodfellow and colleagues in 2014.

A GAN consists of two neural networks, called the generator and the discriminator, which are trained in opposition to one another. Their competition encourages the generator to produce samples that resemble real training data.

GANs have been used for image synthesis, image translation, super-resolution, style transfer, and other generative tasks.

3.1 Main Components of a GAN

3.1.1 Generator

The generator, denoted by G, receives a random latent vector z and transforms it into a synthetic sample.

The latent vector is commonly drawn from a simple probability distribution, such as a Gaussian distribution or a uniform distribution.

The generator's objective is to produce samples that the discriminator identifies as real.

3.1.2 Discriminator

The discriminator, denoted by D, receives either a real sample from the training dataset or a generated sample from the generator.

It estimates whether the input resembles a real training example. In the original GAN formulation, its output can be interpreted as the probability that a sample is real.

The discriminator learns to distinguish real data from synthetic data, while the generator attempts to make that distinction difficult.

3.2 How GANs Work

The training process involves alternating optimization.

1. The generator receives random latent vectors and produces synthetic samples.
2. The discriminator evaluates real training examples and generated samples.

3. The discriminator is trained to distinguish real samples from generated samples.
4. The generator is trained to produce samples that cause the discriminator to classify them as real.
5. These updates continue iteratively, improving the competing networks according to the training objective.

3.3 Mathematical Formulation

The original GAN objective is expressed as a minimax game :

$$\min_G \max_D V(D,G) = E_{x\sim p_{\text{data}}(x)} \left[\log D(x)\right] + E_{z\sim p_z(z)} \left[\log(1 - D(G(z)))\right].$$

In this equation :

- $p_{\text{data}}(x)$ is the real data distribution.
- $p_z(z)$ is the distribution used to sample latent vectors.
- $G(z)$ is a generated sample.
- $D(x)$ is the discriminator's estimate that the sample x is real.

The discriminator attempts to maximize the objective by correctly distinguishing real samples from generated ones. The generator attempts to minimize the objective by producing samples that are difficult to distinguish from real data.

In practice, the generator is often trained using a non-saturating alternative loss to improve gradient behavior. The exact objective may also vary in different GAN architectures.

3.4 Applications of GANs

- **Image synthesis :** Generating realistic faces, objects, and scenes.
- **Image-to-image translation :** Converting sketches into images or translating between visual domains.
- **Super-resolution :** Reconstructing sharper images from lower-resolution inputs.
- **Data augmentation :** Producing synthetic samples for selected machine learning tasks.
- **Style transfer :** Transforming images into different visual styles.
- **Research :** Studying generative modeling, representation learning, and synthetic data generation.

3.5 Advantages and Limitations

GANs can generate visually realistic samples and may produce images efficiently after training. Some architectures offer fine-grained control over particular visual attributes.

However, GAN training can be unstable. A common problem is *mode collapse*, in which the generator produces a limited variety of samples instead of representing the diversity of the training distribution. Training can also be sensitive to the balance between the generator and discriminator.

GANs may reproduce biases or sensitive patterns present in training data. Their outputs therefore require evaluation for quality, privacy, fairness, and suitability for the intended application.

4 What Are Transformers ?

The Transformer is a neural network architecture introduced in the 2017 research paper *Attention Is All You Need*. It relies primarily on attention mechanisms rather than recurrence to model relationships between elements in a sequence.

Transformers have become a foundational architecture for large language models and are also used in vision, audio, multimodal learning, and other areas of machine learning.

GPT is an example of a model family built around a Transformer-based language modeling approach.

4.1 Why Transformers Were Developed

Earlier sequence models often used recurrent neural networks (RNNs) or long short-term memory networks (LSTMs). These architectures process sequence elements sequentially, which can limit parallelization during training.

Transformers allow many token representations to be processed in parallel during training. Their attention mechanisms also allow each token to incorporate information from other relevant positions in the sequence.

This makes Transformers effective for modeling relationships between words or other sequence elements, including relationships that span long distances.

4.2 Main Components of a Transformer

4.2.1 Token Embeddings

The input sequence is first divided into tokens. Each token is mapped to a numerical vector known as an embedding.

These vectors provide a representation of the input that can be processed by neural network layers.

4.2.2 Positional Information

Attention alone does not inherently encode the order of tokens. Therefore, Transformer systems incorporate positional information using methods such as positional embeddings, relative position representations, or rotary position embeddings.

This information helps the model distinguish sequences whose tokens occur in different orders.

4.2.3 Self-Attention

Self-attention enables a token representation to incorporate information from other tokens in the same sequence.

The mechanism constructs three representations :

- **Query (Q) :** Represents what information a token is seeking.
- **Key (K) :** Represents features used to determine the relevance of another token.
- **Value (V) :** Contains the information combined according to the attention weights.

The scaled dot-product attention operation is

$$Attention(Q, K, V) = softmax\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

where d_k is the dimensionality of the key vectors.

The matrix product $QK^\top$ measures relationships between queries and keys. Scaling by $\sqrt{d_k}$ helps control the magnitude of the scores, and softmax converts the scores into normalized attention weights.

These weights determine how information from the value vectors contributes to each output representation.

4.2.4 Multi-Head Attention

Instead of computing attention only once, a Transformer can compute several attention operations in parallel. This is called multi-head attention.

Each head learns a different projection of the input and may capture different relationships. Their outputs are combined to form a richer representation.

4.2.5 Feed-Forward Network

After attention, each token representation is typically processed by a feed-forward neural network. This network applies learned transformations and nonlinear activation functions to the representation.

4.2.6 Residual Connections and Normalization

Transformer blocks commonly include residual connections and normalization layers. Residual connections help preserve information across layers and support optimization, while normalization helps stabilize the processing of intermediate representations.

The precise ordering and normalization method depend on the architecture.

4.3 Encoder and Decoder Architectures

Transformers can be organized in different ways.

- **Encoder-only models :** Use bidirectional contextual representations and are often suitable for classification, retrieval, and text understanding.
- **Decoder-only models :** Commonly use causal attention, allowing each token to attend only to preceding tokens. GPT-style language models use this general approach.
- **Encoder-decoder models :** Use an encoder to represent an input and a decoder to generate an output. These models are useful for tasks such as translation and sequence-to-sequence generation.

4.4 Causal Attention

In autoregressive language generation, a token should not access future target tokens during training. A causal mask prevents attention to positions that occur later in the sequence.

This allows the model to learn to predict the next token using only the permitted preceding context.

For example, when generating the sentence *"The cat is sleeping"*, the model predicts each next token based on the available preceding tokens.

4.5 Applications of Transformers

- Natural-language understanding and generation.
- Machine translation and summarization.
- Conversational assistants and question answering.
- Source-code generation and program analysis.
- Image classification and visual understanding.
- Multimodal applications involving text, images, audio, and other data types.
- Document retrieval and semantic search.

4.6 Advantages and Limitations

Transformers are effective at learning complex relationships in large datasets and support parallel computation during training. Their flexible architecture has enabled progress in language modeling and multimodal AI.

However, standard full self-attention has computational and memory requirements that can grow quadratically with sequence length. Long-context processing can therefore be expensive. Transformers also require substantial data and compute for large-scale training, and their outputs can reflect biases or incorrect patterns learned from the training data.

5 Applications in Enterprise AI

Enterprise AI refers to the use of artificial intelligence within organizations to improve business processes, support decisions, automate tasks, and create new products and services.

Generative AI extends traditional enterprise AI by enabling systems to create text, code, images, summaries, and other outputs from instructions and contextual information.

Organizations can combine generative models with internal databases, business applications, document repositories, workflow tools, and human review processes.

5.1 Customer Service and Support

Generative AI can assist customer service teams by answering frequently asked questions, summarizing conversations, drafting responses, and retrieving relevant product information.

For example, a support assistant can retrieve an approved troubleshooting procedure and generate a response tailored to a customer's question.

When connected to enterprise systems, the assistant may also help agents summarize previous interactions or prepare case notes.

Important controls include access restrictions, escalation to human agents, response evaluation, and verification of information before consequential actions are taken.

5.2 Document Processing and Knowledge Management

Organizations maintain large quantities of documents, including policies, contracts, technical manuals, reports, and meeting notes. Generative AI can help employees search, summarize, compare, and interpret these materials.

Common applications include :

- Summarizing long reports and meetings.
- Extracting important information from contracts.
- Answering questions about internal policies.
- Preparing executive summaries.
- Identifying differences between document versions.

– Organizing and classifying business documents.

A common implementation combines a language model with retrieval-augmented generation (RAG). In a RAG system, relevant information is retrieved from an approved knowledge source and supplied to the model as context.

A simplified workflow is

$$Question \rightarrow Retrieval \rightarrow Context \rightarrow Generation \rightarrow Verification.$$

Retrieval can improve grounding in current organizational information, but it does not guarantee correctness. The retrieved sources, permissions, and generated answers must still be evaluated.

5.3 Software Development and IT Operations

Code-focused models can assist software engineering teams with code generation, debugging, testing, refactoring, and documentation.

Possible enterprise applications include :

– Generating boilerplate code and prototypes.

– Explaining legacy applications.

– Drafting automated tests.

– Suggesting fixes for programming errors.

– Preparing code documentation.

– Helping developers understand application programming interfaces (APIs).

In IT operations, language models may also summarize alerts, explain log messages, and help engineers investigate incidents.

Organizations should integrate these capabilities into controlled development environments. Code review, testing, dependency management, and security scanning remain necessary.

5.4 Human Resources and Employee Productivity

Generative AI can help human resources teams prepare job descriptions, summarize employee surveys, draft training material, and answer routine questions about workplace policies.

Internal assistants may help employees locate information about benefits, leave policies, or onboarding procedures.

However, AI systems used in recruitment, performance evaluation, promotion, or other employment decisions require particular care. Organizations should evaluate fairness, privacy, transparency, applicable law, and the potential impact on employees. Automated recommendations should not replace appropriate human judgment.

5.5 Finance and Business Analysis

Generative AI can support finance and business teams by summarizing reports, explaining variances, generating analytical code, and assisting with financial documentation.

Examples include :

- Summarizing financial statements.
- Explaining budget variances.
- Drafting management reports.
- Supporting natural-language queries over approved financial datasets.
- Generating code for data analysis.

Financial calculations should be performed using validated computational tools rather than relying solely on generated text. Outputs should be reconciled against authoritative financial records, and access to confidential information must be restricted.

5.6 Supply Chain and Manufacturing

Generative AI can assist supply chain teams by summarizing supplier reports, explaining operational incidents, and preparing planning documents.

In manufacturing, AI assistants can help workers locate approved maintenance procedures and interpret technical documentation.

Generative systems can also support the development of synthetic training examples and help engineers document quality issues.

Operational decisions involving machinery, safety systems, inventory commitments, or production schedules should be validated against reliable data and established controls.